

Computer Sciences I

Semester 2026-III

Course Project Description

Eng. Carlos Andrés Sierra, M.Sc.

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Overview

The course project of *Computer Sciences I* is a semester-long, team-based game development project where each team builds a **two-player videogame with computer-controlled logic**. The opponent logic is implemented using fundamental data structures and algorithms learned throughout the semester: binary trees, linked lists, queues, stacks, greedy algorithms, divide-and-conquer strategies, and backtracking.

This project is the practical culmination of the course: every data structure and algorithm studied becomes a building block in your game's opponent decision-making system. The game mechanics are the vehicle; the focus is on correctly implementing and applying the data structures and algorithms to solve real problems.

Project Scope and Philosophy

- i. **Data Structure and Algorithm Implementation First:** The core deliverable is correct, efficient implementation of the required data structures and algorithms — not graphics or gameplay polish.
- ii. **Algorithmic Problem Solving:** The choice of data structures and algorithms directly determines the solution's efficiency and correctness. Teams must understand and justify their design choices.
- iii. **Reproducible and Fair:** All teams work on problems of equivalent complexity. The data structure and algorithm implementation, not the game theme, determines grading.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavir-guezs@udistrital.edu.co*.

- iv. **Complete Deliverable:** The final submission must be a fully functional, stand-alone game running in `Pygame` with no external dependencies beyond the standard library.

The 7 Game Variants

Each team is assigned one unique two-player game. All games share the same grading rubric and are evaluated on correct data structure and algorithm implementation, code organization, and documentation. The list below describes each game and its required data structures/algorithms.

1. Tic-Tac-Toe — Binary Tree + Backtracking

- *Game Mechanics:* Players alternate placing marks (X or O) on a 3×3 grid. First to achieve three in a row (horizontal, vertical, or diagonal) wins.
- *Data Structures and Algorithms:* The computer opponent uses a **binary tree** to represent the game state space and **backtracking** to explore all possible move sequences. Each node represents a board state; the algorithm recursively tries moves, evaluates outcomes, and backtracks to explore alternative branches.
- *Implementation Focus:* Binary tree node construction and traversal, board state representation, win-condition checking, recursive backtracking through the game tree, and move evaluation.
- *Complexity:* Straightforward to implement; excellent for learning tree structures and backtracking fundamentals.

2. Rock Paper Scissors — Linked List for History Tracking

- *Game Mechanics:* Players simultaneously choose Rock, Paper, or Scissors. Best-of-five format: first to win three rounds wins the match.
- *Data Structures and Algorithms:* The computer maintains a **linked list** of the player's historical choices across all rounds. It traverses this list to detect patterns (e.g., player always plays Rock after losing) and exploits them by playing the counter-choice.
- *Implementation Focus:* Linked list node creation and insertion, traversal for pattern detection, choice history management, and decision logic based on observed patterns.
- *Complexity:* Moderate; requires understanding linked list operations and detecting sequential patterns in data.

3. Connect Four — Divide and Conquer Strategy

- *Game Mechanics:* Players drop colored pieces into a 7×6 grid. Gravity pulls pieces down. First to achieve four in a row wins.
- *Data Structures and Algorithms:* The computer opponent uses **divide and conquer** to evaluate moves. It partitions the board into regions, evaluates each region independently for threats and opportunities, combines the evaluations, and selects the best overall move.
- *Implementation Focus:* Column-based piece mechanics, gravity simulation, four-in-a-row detection, board partitioning strategy, region evaluation, and divide-and-conquer decision making.

- *Complexity*: Challenging; requires designing effective board partitioning schemes and combining sub-problem solutions correctly.

4. Checkers — Queue and Stack for Move Management

- *Game Mechanics*: Simplified checkers on an 8×8 board. Pieces move diagonally, capture opponents by jumping, and promote to kings upon reaching the opposite end.
- *Data Structures and Algorithms*: The computer uses a **queue** to manage possible moves in breadth-first order and a **stack** to track move sequences for lookahead evaluation. It evaluates multiple move sequences and selects the one with the best outcome.
- *Implementation Focus*: Piece movement validation, capture detection and execution, queue-based move enumeration, stack-based move sequence tracking, and move selection based on lookahead evaluation.
- *Complexity*: High; requires managing complex game rules, piece types, and correctly implementing queue/stack operations for multi-move lookahead.

5. Memory Game — Doubly Linked List for Card Management

- *Game Mechanics*: Cards are arranged face-down in a grid. Players take turns flipping two cards trying to find matching pairs. A player keeps their turn if they find a match; otherwise play passes to the opponent. Most matches wins.
- *Data Structures and Algorithms*: The computer maintains a **doubly linked list** of all cards revealed, allowing bidirectional traversal through card history. This enables efficient pattern recognition and memory of card positions for optimal pair matching.
- *Implementation Focus*: Doubly linked list node creation with forward and backward pointers, card position tracking using the list, efficient traversal in both directions for pattern detection, and optimal pair identification.
- *Complexity*: Straightforward; excellent for learning doubly linked list operations and bidirectional traversal.

6. Snake Battle — Queue-Based Pathfinding with Greedy Strategy

- *Game Mechanics*: Two snakes move simultaneously on a grid. Food spawns randomly. Snakes eat food to grow; colliding with walls or each other causes elimination. Last snake alive wins.
- *Data Structures and Algorithms*: The computer's snake uses a **queue** to implement breadth-first pathfinding to locate food while avoiding obstacles. It combines this with a **greedy strategy** to always move toward the nearest food and maximize growth potential.
- *Implementation Focus*: Collision detection, queue-based breadth-first pathfinding, greedy nearest-food targeting, obstacle avoidance, independent planning per game cycle, and simultaneous move handling.

- *Complexity*: Moderate to high; requires real-time queue-based pathfinding and handling simultaneous, non-turn-based movement.

7. Pong — Backtracking for Optimal Paddle Position

- *Game Mechanics*: Classic arcade pong: a ball bounces between two paddles on opposite sides of the court. Players score when the opponent fails to return the ball. First to reach a target score wins.
- *Data Structures and Algorithms*: The computer's paddle uses **backtracking** to explore possible paddle positions and trajectories. For each ball state, it tries different positions, simulates the outcome, and backtracks if the position fails to result in a successful return.
- *Implementation Focus*: Physics-based ball movement, collision detection with paddles and walls, ball velocity tracking, backtracking through position candidates, trajectory simulation, and optimal position selection.
- *Complexity*: Moderate; requires understanding physics simulation and implementing backtracking logic to explore the solution space of valid paddle positions.

Technical Requirements

All teams must follow these requirements:

- **Language**: Python 3.9+ with Pygame for all UI rendering.
- **Dependencies**: Only pygame and the Python standard library. No external libraries, frameworks, or utilities beyond these.
- **Entry Point**: The game must run from a single command: `python main.py` (or equivalent).
- **Algorithm and Data Structure Implementation**: The opponent logic implementing the required data structures and algorithms must be clearly separated from game logic and UI. The opponent module must be independently testable.
- **Code Organization**: Recommended structure:
 - `main.py` — Entry point and game loop
 - `game.py` — Game state, rules, and win conditions
 - `opponent.py` — Data structures, algorithms, and opponent decision making
 - `ui.py` — Pygame rendering and input handling
 - `tests.py` (optional) — Unit tests for opponent logic and game state
- **Documentation**: Header comments in all files with team number, game name, and student names.
- **GitHub**: Complete repository with version control history demonstrating iterative development.

Checkpoint Submission (Week 8)

At the midpoint of the semester, each team submits a **single PDF document** (no code required) containing:

1. **Game Rules and Mechanics:** Complete, unambiguous description of how the game is played. Include winning conditions, special rules, and edge cases.
2. **Data Structures and Algorithm Pseudocode:** Detailed pseudocode for the data structures and algorithms required in your game. Include all major decision points, data structure operations, and traversal logic.
3. **Board/State Representation:** Diagram or description of how game state will be represented in code (data structures used, class hierarchy, algorithm integration).
4. **Pygame Wireframe:** Hand-drawn or digital sketch of the game UI showing the game board, score display, player/opponent indicators, and any menus or overlays.
5. **Work Split Statement:** A paragraph or table declaring which team member will focus on which components (opponent logic/data structures, game logic, UI) and when.
6. **Algorithm and Data Structure Justification:** Why are the chosen data structures and algorithms appropriate for this game? How do they contribute to the opponent's decision-making?

Final Submission (Week 16)

The team must deliver:

- **Fully Playable Game:** A working, bug-free game where a human player can compete against the AI opponent. The game must be completable (winnable or loseable) through multiple plays.
- **GitHub Repository:** A well-organized public or shared repository with:
 - Complete source code with clear structure
 - Comprehensive `README.md` with setup instructions, gameplay rules, and how to run the game
 - Commit history showing iterative development (not a single “final” commit)
 - Optional: example gameplay logs or test outputs demonstrating AI behavior
- **Technical Report (PDF):** A document covering:
 - *Data Structure and Algorithm Description:* Detailed explanation of the required data structures and algorithms with pseudocode and complexity analysis (time and space).

- *Implementation Details*: Description of data structure implementations, game state management, and how the opponent logic integrates with game rules.
 - *Design Decisions*: Justification for architectural choices (class hierarchy, state representation, data structure selection, algorithm integration points).
 - *Results and Testing*: Description of test scenarios (opponent vs. opponent plays, human vs. opponent multiple matches) and observations about opponent performance.
 - *Challenges and Solutions*: Technical obstacles encountered during development and how they were resolved.
 - *Lessons Learned*: Reflection on data structure and algorithm implementation, debugging process, and professional software development practices.
- **Live In-Class Demo**: Each team demonstrates the game during the final session (Week 16), playing multiple rounds against the opponent logic. All team members must be present and able to explain any component of the project.

Evaluation Criteria

- **Data Structure and Algorithm Implementation (40%)**: Correctness of data structure operations, proper algorithm implementation, complexity awareness, and decision-making quality. The opponent logic should demonstrate mastery of the required data structures and algorithms.
- **Code Quality (30%)**: Clarity, organization, documentation, and adherence to requirements. Code must be readable, maintainable, and properly separate data structure/algorithm logic from game logic.
- **Game Completeness (20%)**: Game is fully playable, all rules are implemented correctly, win conditions work, and UI is functional and responsive.
- **Documentation (10%)**: Technical report clarity, README completeness, and code comments. Demonstration of understanding of data structures, algorithms, and the overall project.

Academic Integrity and Plagiarism

- All code must be written by the team members. Code copied from external sources, tutorials, or templates without substantial original implementation is considered academic dishonesty.
- Plagiarism detection tools will be used to check for code similarity across all submissions. Teams with matching implementations will receive a zero on the affected components.

- Each student is individually responsible for understanding the entire project, not just their assigned role. Every team member must understand the data structures and algorithms used.
- During the final demo, each team member will be questioned individually about the data structures, algorithms, and implementation choices. Lack of understanding will result in a reduced grade for that student.

This project is your opportunity to implement real data structures and algorithms — not a tutorial, not a template, but a functional system that solves real problems within a game context. The satisfaction of building an opponent that competes against you using only the data structures and algorithms you wrote is worth every hour invested. Build it well, play it often, and enjoy the journey from algorithmic concept to interactive reality!